

**INSTITUTO
FEDERAL**
Santa Catarina

Sintaxe do JavaScript - pt. 1

Programação Frontend I

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



INSTITUTO FEDERAL

Santa Catarina

Câmpus São José

PENSAMENTO
COMPUTACIONAL E
ALGORITMOS

Análise e Desenvolvimento
de Sistemas

Inserindo JS no Documento HTML

scripts incorporados

- inserindo o código dentro de `<script>..</script>`
- apenas o documento que o contém é afetado
- pode estar dentro do elemento `<head>` ou ao final do elemento `<body>`



Inserindo JS no Documento HTML

scripts incorporados

- inserindo o código dentro de `<script>..</script>`
- apenas o documento que o contém é afetado
- pode estar dentro do elemento `<head>` ou ao final do elemento `<body>`

exemplo

```
<script>  
  alert("Olá mundo!");  
</script>
```



Inserindo JS no Documento HTML

scripts externos

- documento separado contendo o código
- arquivo texto com extensão **.js**
- importado com o elemento `<script>` com o atributo `src="..."`
- vários documentos HTML podem usar o mesmo script



Inserindo JS no Documento HTML

scripts externos

- documento separado contendo o código
- arquivo texto com extensão **.js**
- importado com o elemento `<script>` com o atributo `src="..."`
- vários documentos HTML podem usar o mesmo script

exemplo

```
<script src="../../script/exemplo.js"></script>
```



Tipos de Dados

exemplos

■ Indefinido (undefined)

- variável declarada, mas sem valor atribuído

```
var idade;
```

Tipos de Dados

exemplos

- Indefinido (undefined)
- **Nulo (null)**
 - similar a undefined

```
var idade;
```

```
var idade = null;
```

Tipos de Dados

exemplos

- Indefinido (undefined)
- Nulo (null)
- **Número (number)**
 - qualquer valor numérico

```
var idade;
```

```
var idade = null;
```

```
var idade = 20;
```



Tipos de Dados

- Indefinido (undefined)
- Nulo (null)
- Número (number)
- **String**
 - um trecho de texto

exemplos

```
var idade;
```

```
var idade = null;
```

```
var idade = 20;
```

```
var idade = "vinte";
```



Tipos de Dados

exemplos

- Indefinido (undefined)
- Nulo (null)
- Número (number)
- String
- Booleanos (boolean)
 - valores lógicos "falso" ou "verdadeiro"

```
var idade;
```

```
var idade = null;
```

```
var idade = 20;
```

```
var idade = "vinte";
```

```
var idade = true;
```



Tipos de Dados

- Indefinido (undefined)
- Nulo (null)
- Número (number)
- String
- Booleanos (boolean)
 - falso ("*falsy*")
 - verdadeiro ("*truthy*")

exemplos

```
var idade;
```

```
var idade = null;
```

```
var idade = 20;
```

```
var idade = "vinte";
```

```
var idade = true;
```

```
null, undefined, 0, ""
```

todo o resto



Operadores Relacionais

exemplos

■ == é igual a

```
alert("5" == 5);
```

Operadores Relacionais

exemplos

■ == é igual a

```
alert("5" == 5); // true
```

Operadores Relacionais

exemplos

- == é igual a
- != não é igual a

```
alert("5" == 5); // true
```

```
alert(5 != 6);
```

Operadores Relacionais

exemplos

- == é igual a
- != não é igual a

```
alert("5" == 5); // true
```

```
alert(5 != 6); // true
```

Operadores Relacionais

exemplos

- == é igual a
- != não é igual a
- === é idêntico a (igual e do mesmo tipo)

```
alert("5" == 5); // true
```

```
alert(5 != 6); // true
```

```
alert("5" === 5);
```



Operadores Relacionais

exemplos

- == é igual a
 - != não é igual a
 - === é idêntico a (igual e do mesmo tipo)
- ```
alert("5" == 5); // true
alert(5 != 6); // true
alert("5" === 5); // false
```



# Operadores Relacionais

## exemplos

- ==      é igual a
- !=      não é igual a
- ===    é idêntico a (igual e do mesmo tipo)
- !==    não é idêntico a

```
alert("5" == 5); // true
```

```
alert(5 != 6); // true
```

```
alert("5" === 5); // false
```

```
alert("5" !== 5);
```



# Operadores Relacionais

## exemplos

- ==      é igual a
- !=      não é igual a
- ===    é idêntico a (igual e do mesmo tipo)
- !==    não é idêntico a

```
alert("5" == 5); // true
```

```
alert(5 != 6); // true
```

```
alert("5" === 5); // false
```

```
alert("5" !== 5); // true
```



# Operadores Relacionais

## exemplos

- ==      é igual a
- !=      não é igual a
- ===    é idêntico a (igual e do mesmo tipo)
- !==    não é idêntico a
- >      é maior do que

```
alert("5" == 5); // true
```

```
alert(5 != 6); // true
```

```
alert("5" === 5); // false
```

```
alert("5" !== 5); // true
```

```
alert(5 > 4); // true
```



# Operadores Relacionais

## exemplos

- ==      é igual a
- !=      não é igual a
- ===    é idêntico a (igual e do mesmo tipo)
- !==    não é idêntico a
- >      é maior do que
- >=     é maior do que ou igual a

```
alert("5" == 5); // true
```

```
alert(5 != 6); // true
```

```
alert("5" === 5); // false
```

```
alert("5" !== 5); // true
```

```
alert(5 > 4); // true
```

```
alert(5 >= 5);
```



# Operadores Relacionais

## exemplos

- ==      é igual a
- !=      não é igual a
- ===    é idêntico a (igual e do mesmo tipo)
- !==    não é idêntico a
- >      é maior do que
- >=     é maior do que ou igual a

```
alert("5" == 5); // true
```

```
alert(5 != 6); // true
```

```
alert("5" === 5); // false
```

```
alert("5" !== 5); // true
```

```
alert(5 > 4); // true
```

```
alert(5 >= 5); // true
```



# Operadores Relacionais

## exemplos

- ==      é igual a
- !=      não é igual a
- ===    é idêntico a (igual e do mesmo tipo)
- !==    não é idêntico a
- >      é maior do que
- >=     é maior do que ou igual a
- <      é menor do que

```
alert("5" == 5); // true
```

```
alert(5 != 6); // true
```

```
alert("5" === 5); // false
```

```
alert("5" !== 5); // true
```

```
alert(5 > 4); // true
```

```
alert(5 >= 5); // true
```

```
alert(5 < 6); // true
```

# Operadores Relacionais

## exemplos

- ==      é igual a
- !=      não é igual a
- ===    é idêntico a (igual e do mesmo tipo)
- !==    não é idêntico a
- >      é maior do que
- >=     é maior do que ou igual a
- <      é menor do que
- <=     é menor do que ou igual a

```
alert("5" == 5); // true
```

```
alert(5 != 6); // true
```

```
alert("5" === 5); // false
```

```
alert("5" !== 5); // true
```

```
alert(5 > 4); // true
```

```
alert(5 >= 5); // true
```

```
alert(5 < 6); // true
```

```
alert(5 <= 5); // true
```



# Variáveis

- contêiner de informação

# Variáveis

- contêiner de informação
- um número, string de texto, um elemento no DOM ou uma função

# Variáveis

- contêiner de informação
- um número, string de texto, um elemento no DOM ou uma função
- a declaração começa com **var**, **let** ou **const**

## exemplo

```
var idade = 20;
let distancia = 150;
const versao = 1.2;
```



# Variáveis

- contêiner de informação
- um número, string de texto, um elemento no DOM ou uma função
- a declaração começa com **var**, **let** ou **const**

## exemplo

```
var idade = 20;
let distancia = 150;
const versao = 1.2;
```

- nomes das variáveis:
  - devem começar com uma letra ou um sublinhado (\_)
  - podem conter letras, dígitos e sublinhados em qualquer combinação
  - não podem conter espaços
  - não podem conter caracteres especiais (! . , / \ + \* =)



# Variáveis

## declaração e atribuição

## exemplos

- **var** escopo de função ou global

# Variáveis

## declaração e atribuição

- **var** escopo de função ou global

## exemplos

```
var a;
var b = 6;
```

# Variáveis

## declaração e atribuição

- **var** escopo de função ou global

## exemplos

```
var a;
var b = 6;
```

```
var nome = "Ana"; // Variável global

function mostrarNome() {
 console.log(nome); // Acessa a variável global dentro da função
}

mostrarNome(); // Saída: "Ana"
console.log(nome); // Saída: "Ana"
```



# Variáveis

## declaração e atribuição

- **var** escopo de função ou global

## exemplos

```
var a;
var b = 6;
```

```
function exemplo() {
 var mensagem = "Olá!";
 console.log(mensagem); // Saída: "Olá!"
}

exemplo(); // Chama a função

console.log(mensagem); // ✗ Erro! "mensagem" só existe dentro da função
```



# Variáveis

## declaração e atribuição

- **var** escopo de função ou global
- **let** escopo de bloco

## exemplos

```
var a;
var b = 6;
```

```
if (true) {
 let idade = 25; // Variável com escopo de bloco
 console.log(idade); // Saída: 25
}

console.log(idade); // ✗ Erro! "idade" só existe dentro do bloco '{}'.
}
```

# Variáveis

## declaração e atribuição

- **var** escopo de função ou global
- **let** escopo de bloco

## exemplos

```
var a;
var b = 6;
```

```
let c;
let d, soma;
let i = 0, j = 0, k = 0;
```



# Variáveis

## declaração e atribuição

- **var** escopo de função ou global
- **let** escopo de bloco
- **const** valores não mudam (constantes)

## exemplos

```
var a;
var b = 6;
```

```
let c;
let d, soma;
let i = 0, j = 0, k = 0;
```



# Variáveis

## declaração e atribuição

- **var** escopo de função ou global
- **let** escopo de bloco
- **const** valores não mudam (constantes)

## exemplos

```
var a;
var b = 6;
```

```
let c;
let d, soma;
let i = 0, j = 0, k = 0;
```

```
const VERSAO = 1.01;
const C = 299792.458;
```



# Comentários

- **texto ignorado (não executado)**
- **uso** explicações, lembretes ou retirar temporariamente trechos de código
- útil quando o código vai ser editado por outra pessoa no futuro
- tipos de comentários
  - comentários em linha (//)
  - comentários em bloco (/\* . . \*/)

exemplos



# Comentários

- **texto ignorado (não executado)**
- **uso** explicações, lembretes ou retirar temporariamente trechos de código
- útil quando o código vai ser editado por outra pessoa no futuro
- tipos de comentários
  - comentários em linha (//)
  - comentários em bloco (/\* . \*/)

## exemplos

```
// Este é um comentário em linha.
// Ele é desativado a cada fim de
// linha
```

```
/* Este é um comentário em bloco.
Somente será ativado com o símbolo
de fechamento */
```

# Literais

- **valor de dados que aparece diretamente no código**

exemplos

# Literais

- **valor de dados que aparece diretamente no código**
- tipos de literais
  - numéricos

## exemplos

```
12 // número doze
1.2 // número um ponto dois
0xff // número 255 em hexadecimal
1.2e23 // número 1.2×10^{23}
```



# Literais

- **valor de dados que aparece diretamente no código**
- tipos de literais
  - numéricos
  - textuais (strings)

## exemplos

```
12 // número doze
1.2 // número um ponto dois
0xff // número 255 em hexadecimal
1.2e23 // número 1.2 x 10^23

"olá mundo!" // uma string
"olá\nmundo!" // string em duas linhas
"olá\
mundo!" // uma string em duas linhas
```

# Literais

- **valor de dados que aparece diretamente no código**
- tipos de literais
  - numéricos
  - textuais (strings)
  - booleanos

## exemplos

```
12 // número doze
1.2 // número um ponto dois
0xff // número 255 em hexadecimal
1.2e23 // número 1.2 x 10^23

"olá mundo!" // uma string
"olá\nmundo!" // string em duas linhas
"olá\
mundo!" // uma string em duas linhas

true // um valor Booleano
false // outro valor Booleano
```

# Palavras Reservadas

- **palavras que fazem parte da linguagem JS**
- não podem ser usadas para nomear variáveis, constantes, funções, classes, etc.

# Palavras Reservadas

- palavras que fazem parte da linguagem JS
- não podem ser usadas para nomear variáveis, constantes, funções, classes, etc.

## ES6

as async await break case catch class const  
continue debugger default delete do else  
export extends false finally for from function  
get if import in instanceof let new null of  
return set static super switch target this  
throw true try typeof var void while with  
yield



# Palavras Reservadas

- palavras que fazem parte da linguagem JS
- não podem ser usadas para nomear variáveis, constantes, funções, classes, etc.

## ES6

as async await break case catch class const  
continue debugger default delete do else  
export extends false finally for from function  
get if import in instanceof let new null of  
return set static super switch target this  
throw true try typeof var void while with  
yield

## uso futuro

enum implements interface package private  
protected public



# Coleções (Arrays)

- grupo de valores ordenados que podem ser atribuídos a uma única variável

exemplos

- os valores em um array são chamados **elementos**
- a posição numérica dos elementos é chamada de **índice** (começa em 0)
- os elementos podem ser de tipos diferentes em um mesmo array
- arrays são dinâmicos
- arrays podem ser esparsos



# Coleções (Arrays)

- grupo de valores ordenados que podem ser atribuídos a uma única variável
- os valores em um array são chamados **elementos**
- a posição numérica dos elementos é chamada de **índice** (começa em 0)
- os elementos podem ser de tipos diferentes em um mesmo array
- arrays são dinâmicos
- arrays podem ser esparsos

## exemplos

```
let vazio = [];
let cinco = [5, "cinco", true];
let esperso = [1, , 3];
let espersoVazio = [,,,];
```



# Coleções (Arrays)

## operador de propagação ( . . . )

## exemplos

- insere elementos de um array como elementos de outro array





# Coleções (Arrays)

## operador de propagação (...)

- insere elementos de um array como elementos de outro array

## exemplos

```
let a = [1, 2, 3];
let b = [0, ...a, 4];
```



# Coleções (Arrays)

## operador de propagação (...)

- insere elementos de um array como elementos de outro array

## exemplos

```
let a = [1, 2, 3];
let b = [0, ...a, 4];
// b == [0, 1, 2, 3, 4]
```



# Coleções (Arrays)

## operador de propagação (...)

- insere elementos de um array como elementos de outro array
- funciona com qualquer objeto iterável

## exemplos

```
let a = [1, 2, 3];
let b = [0, ...a, 4];
// b == [0, 1, 2, 3, 4]

let Algarismos = [..."0123456789"];
```



# Coleções (Arrays)

## operador de propagação (...)

- insere elementos de um array como elementos de outro array
- funciona com qualquer objeto iterável

## exemplos

```
let a = [1, 2, 3];
let b = [0, ...a, 4];
// b == [0, 1, 2, 3, 4]
```

```
let Algarismos = [..."0123456789"];
// Algarismos == ["0", "1", "2", "3", "4",
"5", "6", "7", "8", "9"]
```



# Coleções (Arrays)

criando arrays com o construtor

exemplos

- `Array()`



# Coleções (Arrays)

## criando arrays com o construtor

- `Array()`
- **sem argumentos** cria um array vazio (similar ao literal `[]`)

## exemplos

```
let a = new Array(); // a == []
```

# Coleções (Arrays)

## criando arrays com o construtor

- `Array()`
- **sem argumentos** cria um array vazio (similar ao literal `[]`)
- **um argumento numérico** especifica o comprimento (`length`)

## exemplos

```
let a = new Array(); // a == []
```

```
let b = new Array(4);
```



# Coleções (Arrays)

## criando arrays com o construtor

- `Array()`
- **sem argumentos** cria um array vazio (similar ao literal `[]`)
- **um argumento numérico** especifica o comprimento (`length`)
- **dois ou mais argumentos numéricos ou pelo menos um não numérico** explicita os elementos do array

## exemplos

```
let a = new Array(); // a == []
```

```
let b = new Array(4);
```

```
let c = new Array(6, "casa", true);
```





# Coleções (Arrays)

## comprimento de arrays

- todo array tem a propriedade `length`

## exemplos



# Coleções (Arrays)

## comprimento de arrays

- todo array tem a propriedade `length`

## exemplos

```
let a = [,,,];
```



# Coleções (Arrays)

## arrays esparsos

- os elementos não têm elementos em índices contíguos

## exemplos

```
let a = [,,,];
```

```
let b = new Array(4);
b[1000] = 0;
```



# Coleções (Arrays)

## comprimento de arrays

- todo array tem a propriedade `length`

## exemplos



# Coleções (Arrays)

## comprimento de arrays

- todo array tem a propriedade `length`

## exemplos

```
[].length // 0
["a", "b", "c"].length // 3
let b = new Array(4);
b.length // 4
```



# Coleções (Arrays)

## comprimento de arrays

- todo array tem a propriedade `length`
- `length` é sempre maior do que qualquer índice

## exemplos

```
[].length // 0
["a", "b", "c"].length // 3
let b = new Array(4);
b.length // 4
```

# Coleções (Arrays)

## comprimento de arrays

- todo array tem a propriedade `length`
- `length` é sempre maior do que qualquer índice

## exemplos

```
[].length // 0
["a", "b", "c"].length // 3
let b = new Array(4);
b.length // 4
b[1000] = 0; // b.length == 1001
```

# Coleções (Arrays)

## comprimento de arrays

- todo array tem a propriedade `length`
- `length` é sempre maior do que qualquer índice

## exemplos

```
[].length // 0
["a", "b", "c"].length // 3
let b = new Array(4);
b.length // 4
b[1000] = 0; // b.length == 1001
a = [1,2,3,4,5]; // a.length == 5
```





# Coleções (Arrays)

## comprimento de arrays

- todo array tem a propriedade `length`
- `length` é sempre maior do que qualquer índice

## exemplos

```
[].length // 0
["a", "b", "c"].length // 3
let b = new Array(4);
b.length // 4
b[1000] = 0; // b.length == 1001
a = [1,2,3,4,5]; // a.length == 5
a.length = 3; // a == [1,2,3]
```



# Coleções (Arrays)

## comprimento de arrays

- todo array tem a propriedade `length`
- `length` é sempre maior do que qualquer índice

## exemplos

```
[].length // 0
["a", "b", "c"].length // 3
let b = new Array(4);
b.length // 4
b[1000] = 0; // b.length == 1001
a = [1,2,3,4,5]; // a.length == 5
a.length = 3; // a == [1,2,3]
a.length = 0; // a == []
```

# Coleções (Arrays)

## comprimento de arrays

- todo array tem a propriedade `length`
- `length` é sempre maior do que qualquer índice

## exemplos

```
[].length // 0
["a", "b", "c"].length // 3
let b = new Array(4);
b.length // 4
b[1000] = 0; // b.length == 1001
a = [1,2,3,4,5]; // a.length == 5
a.length = 3; // a == [1,2,3]
a.length = 0; // a == []
a.length = 5; // sem elementos (esparso)
```



# Coleções (Arrays)

## lendo e escrevendo

- acessando os elementos pelos índices

## exemplos



# Coleções (Arrays)

## lendo e escrevendo

- acessando os elementos pelos índices

## exemplos

```
let a = ["casa"];
let b = a[0];
```



# Coleções (Arrays)

## lendo e escrevendo

- acessando os elementos pelos índices

## exemplos

```
let a = ["casa"];
let b = a[0]; // b == "casa"
```



# Coleções (Arrays)

## lendo e escrevendo

- acessando os elementos pelos índices

## exemplos

```
let a = ["casa"];
let b = a[0]; // b == "casa"
a[1] = 3.14; // escreve o elemento 1
```

# Coleções (Arrays)

## lendo e escrevendo

- acessando os elementos pelos índices

## exemplos

```
let a = ["casa"];
let b = a[0]; // b == "casa"
a[1] = 3.14; // escreve o elemento 1
let i = 2;
a[i] = 3; // escreve o elemento 2
```





# Coleções (Arrays)

## lendo e escrevendo

- acessando os elementos pelos índices

## exemplos

```
let a = ["casa"];
let b = a[0]; // b == "casa"
a[1] = 3.14; // escreve o elemento 1
let i = 2;
a[i] = 3; // escreve o elemento 2
a[i + 1] = "a"; // escreve o elemento 3
```

# Coleções (Arrays)

## lendo e escrevendo

- acessando os elementos pelos índices

## exemplos

```
let a = ["casa"];
let b = a[0]; // b == "casa"
a[1] = 3.14; // escreve o elemento 1
let i = 2;
a[i] = 3; // escreve o elemento 2
a[i + 1] = "a"; // escreve o elemento 3
a[a[i]] = a[0];
```

# Coleções (Arrays)

## lendo e escrevendo

- acessando os elementos pelos índices

## exemplos

```
let a = ["casa"];
let b = a[0]; // b == "casa"
a[1] = 3.14; // escreve o elemento 1
let i = 2;
a[i] = 3; // escreve o elemento 2
a[i + 1] = "a"; // escreve o elemento 3
a[a[i]] = a[0]; /* lê elementos 0 e 2 e escreve
o elemento 3 */
```



# Coleções (Arrays)

## lendo e escrevendo

- acessando os elementos pelos índices

## exemplos

```
let a = ["casa"];
let b = a[0]; // b == "casa"
a[1] = 3.14; // escreve o elemento 1
let i = 2;
a[i] = 3; // escreve o elemento 2
a[i + 1] = "a"; // escreve o elemento 3
a[a[i]] = a[0]; /* lê elementos 0 e 2 e escreve
o elemento 3 */
a["4"] = 0; // escreve o elemento 4
```



# Coleções (Arrays)

adicionando e excluindo elementos

exemplos

- atribuindo valores a novos índices



# Coleções (Arrays)

## adicionando e excluindo elementos

- atribuindo valores a novos índices

## exemplos

```
let a = [];
a[0] = "zero"; // a == ["zero"]
a[1] = "um"; // a == ["zero", "um"]
```

# Coleções (Arrays)

## adicionando e excluindo elementos

- atribuindo valores a novos índices
- **push( )** adiciona valores ao final do array (`a[a.length]`)

## exemplos

```
let a = [];
a[0] = "zero"; // a == ["zero"]
a[1] = "um"; // a == ["zero", "um"]
```



# Coleções (Arrays)

## adicionando e excluindo elementos

- atribuindo valores a novos índices
- **push( )** adiciona valores ao final do array (`a[a.length]`)

## exemplos

```
let a = [];
a[0] = "zero"; // a == ["zero"]
a[1] = "um"; // a == ["zero", "um"]
let b = [];
b.push("zero");
b.push("um", "dois");
```



# Coleções (Arrays)

## adicionando e excluindo elementos

- atribuindo valores a novos índices
- **push()** adiciona valores ao final do array (`a[a.length]`)
- **pop()** remove valores do final do array

## exemplos

```
let a = [];
a[0] = "zero"; // a == ["zero"]
a[1] = "um"; // a == ["zero", "um"]
let b = [];
b.push("zero");
b.push("um", "dois");
```



# Coleções (Arrays)

## adicionando e excluindo elementos

- atribuindo valores a novos índices
- **push()** adiciona valores ao final do array (`a[a.length]`)
- **pop()** remove valores do final do array

## exemplos

```
let a = [];
a[0] = "zero"; // a == ["zero"]
a[1] = "um"; // a == ["zero", "um"]
let b = [];
b.push("zero");
b.push("um", "dois");
b.pop(); // b == ["zero", "um"]
```



# Coleções (Arrays)

## adicionando e excluindo elementos

- atribuindo valores a novos índices
- **push()** adiciona valores ao final do array (`a[a.length]`)
- **pop()** remove valores do final do array
- **unshift()** adiciona valores no início do array e move os outros elementos uma posição para trás

## exemplos

```
let a = [];
a[0] = "zero"; // a == ["zero"]
a[1] = "um"; // a == ["zero", "um"]
let b = [];
b.push("zero");
b.push("um", "dois");
b.pop(); // b == ["zero", "um"]
```



# Coleções (Arrays)

## adicionando e excluindo elementos

- atribuindo valores a novos índices
- **push()** adiciona valores ao final do array (`a[a.length]`)
- **pop()** remove valores do final do array
- **unshift()** adiciona valores no início do array e move os outros elementos uma posição para trás

## exemplos

```
let a = [];
a[0] = "zero"; // a == ["zero"]
a[1] = "um"; // a == ["zero", "um"]
let b = [];
b.push("zero");
b.push("um", "dois");
b.pop(); // b == ["zero", "um"]
b.unshift(-1); // b == [-1, "zero", "um"]
```



# Coleções (Arrays)

## adicionando e excluindo elementos

- atribuindo valores a novos índices
- **push()** adiciona valores ao final do array (`a[a.length]`)
- **pop()** remove valores do final do array
- **unshift()** adiciona valores no início do array e move os outros elementos uma posição para trás
- **shift()** remove valores do início do array e move os outros elementos uma posição para frente

## exemplos

```
let a = [];
a[0] = "zero"; // a == ["zero"]
a[1] = "um"; // a == ["zero", "um"]
let b = [];
b.push("zero");
b.push("um", "dois");
b.pop(); // b == ["zero", "um"]
b.unshift(-1); // b == [-1, "zero", "um"]
```



# Coleções (Arrays)

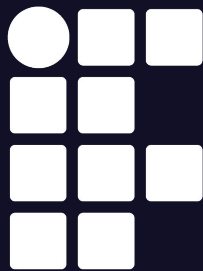
## adicionando e excluindo elementos

- atribuindo valores a novos índices
- **push()** adiciona valores ao final do array (`a[a.length]`)
- **pop()** remove valores do final do array
- **unshift()** adiciona valores no início do array e move os outros elementos uma posição para trás
- **shift()** remove valores do início do array e move os outros elementos uma posição para frente

## exemplos

```
let a = [];
a[0] = "zero"; // a == ["zero"]
a[1] = "um"; // a == ["zero", "um"]
let b = [];
b.push("zero");
b.push("um", "dois");
b.pop(); // b == ["zero", "um"]
b.unshift(-1); // b == [-1, "zero", "um"]
b.shift(); // b == ["zero", "um"]
```





**INSTITUTO  
FEDERAL**  
Santa Catarina

# Sintaxe do JavaScript - pt. 1

## Programação Frontend I

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

[adriano.lima@ifsc.edu.br](mailto:adriano.lima@ifsc.edu.br)



INSTITUTO FEDERAL

Santa Catarina

Câmpus São José

PENSAMENTO  
COMPUTACIONAL E  
ALGORITMOS

Análise e Desenvolvimento  
de Sistemas